

Perfect. Let's perform **Experiment 6 practically** on your system step by step.

What We Are Going to Build

We will create a simple Python application:

```
print("Hello World from Docker Container")
```

Then:

hello.py



Dockerfile



Docker Image (myimage)



Docker Container (pythoncontainer)



Output

This is called **Containerization**.

Step 1: Create Project Folder

Open **Command Prompt (CMD)**.

Go to Desktop:

```
cd Desktop
```

Create a folder:

```
mkdir DockerDemo
```

Enter the folder:

```
cd DockerDemo
```

Check your location:

```
cd
```

Expected:

```
C:\Users\malan\Desktop\DockerDemo
```

Step 2: Create Python File

In the same CMD:

```
notepad hello.py
```

If Notepad asks:

Do you want to create a new file?

Click **Yes**.

Paste:

```
print("Hello World from Docker Container")
```

Save and close Notepad.

Step 3: Verify File Creation

Run:

```
dir
```

Expected:

```
hello.py
```

Step 4: Create Dockerfile

In CMD:

```
notepad Dockerfile
```

Paste:

```
FROM python:3.9
```

```
COPY hello.py /app/
```

```
WORKDIR /app/
```

```
CMD ["python", "hello.py"]
```

Save and close.

What does this Dockerfile do?

FROM python:3.9

Uses Python image as base.

COPY hello.py /app/

Copies your Python file into the image.

WORKDIR /app/

Sets /app as working directory.

CMD ["python", "hello.py"]

Runs the program when the container starts.

Step 5: Verify Both Files Exist

Run:

dir

Expected:

Dockerfile

hello.py

Step 6: Build Docker Image

Still inside:

C:\Users\malan\Desktop\DockerDemo

Run:

docker build -t myimage .

⚠ Don't forget the dot (.) at the end.

Docker will:

1. Download Python image
2. Read Dockerfile
3. Copy hello.py
4. Create image named myimage

Expected ending:

Successfully tagged myimage:latest

Step 7: Verify Image

Run:

docker images

You should now see:

REPOSITORY

myimage

ubuntu

hello-world

What happened?

Before:

hello-world

ubuntu

Now:

hello-world

ubuntu

myimage

You created your own image.

Step 8: Run Container

Run:

docker run --name pythoncontainer myimage

Output:

Hello World from Docker Container

Congratulations 🎉

Your application is now running inside a Docker container.

Step 9: Verify Container

Run:

```
docker ps -a
```

Expected:

```
pythoncontainer
```

Status:

```
Exited (0)
```

This means the program executed successfully and ended.

Step 10: View Logs

Run:

```
docker logs pythoncontainer
```

Output:

```
Hello World from Docker Container
```

This is the verification step required in your experiment sheet.

What You Have Created

```
hello.py
```

↓

```
Dockerfile
```

↓

```
docker build
```

↓

```
myimage
```

↓

```
docker run
```

↓

```
pythoncontainer
```

↓

```
Hello World from Docker Container
```

Screenshots for Record Book

Take screenshots of:

1.

```
dir
```

(showing Dockerfile and hello.py)

2.

```
docker build -t myimage .
```

3.

```
docker images
```

(showing myimage)

4.

```
docker run --name pythoncontainer myimage
```

(showing Hello World output)

5.

```
docker ps -a
```

6.

```
docker logs pythoncontainer
```

Viva Questions

What is Containerization?

Packaging an application with its dependencies into a container.

What is Dockerfile?

A text file containing instructions to build a Docker image.

What is docker build?

Creates an image from a Dockerfile.

What is docker run?

Creates and starts a container from an image.

What is the purpose of docker logs?

Displays container output and logs.

Now start with **Steps 1–5**, then send me the output of:

`dir`

showing hello.py and Dockerfile, and I'll verify everything before you build the image.